To state this formally, the GRPO objective is very similar to the PPO objective above:

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}A_i, \text{clip}\left(\frac{\pi_\theta(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta D_{KL}(\pi_\theta||\pi_{ref})\right).$$

Note that relative to PPO, the standard implementation of GRPO includes the KL distance in the loss. With the advantage computation for the completion index i:

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \cdots, r_G)}{\text{std}(r_1, r_2, \cdots, r_G)}. \quad (4)$$

$$D_{\text{KL}}\left[\pi_\theta \mid \pi_{\text{ref}}\right] = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - \log\frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - 1,$$

Figure 1: Background schematic for GRPO. This figure is included only for context. The contribution of the repository analyzed in the main paper is not a new GRPO loss, but a staged controller around an existing GRPO trainer.

SUPPLEMENTARY MATERIAL FOR A CASE STUDY OF STAGED METRIC-GATED GRPO FOR VISUAL NUMERIC REASONING

Anonymous authors
Paper under double-blind review

1 PURPOSE OF THE SUPPLEMENT

This supplement expands the main case-study report in four directions:

1. the exact reward and evaluation formulas used by the repository,
2. the controller rules and checkpoint-selection logic,
3. the low-memory runtime profile used by the archived successful runs, and
4. the continuation semantics used for same-phase resume versus cross-phase warm start.

All statements below are grounded in the following repository modules:

- `staged_rl/config.py`
- `staged_rl/rewarding.py`
- `staged_rl/evaluation.py`
- `staged_rl/controller.py`
- `staged_rl/checkpointing.py`
- `staged_rl/trainer_runtime.py`

2 BACKGROUND: GRPO VERSUS THE CONTRIBUTION OF THIS PAPER

The repository uses TRL's `GRPOTrainer` as its optimization backend. The substantive contribution studied in the main report lies outside the core GRPO update:

- staged curriculum construction over filtered MathVista subsets,
- a structured response contract,
- metric-gated reward reweighting after checkpoint evaluation, and
- alias-based checkpoint selection for continuation.

```
total reward
= correctness_weight × correctness_reward
+ formatting_weight × formatting_reward
+ parseability_weight × parseability_reward
+ finished_weight × finished_reward
+ tolerance_weight × tolerance_reward
+ brevity_weight × brevity_reward
```

Figure 2: Compact visual summary of the reward decomposition used in the repository. The equations below give the exact code-aligned definition.

We make this distinction explicit because readers can easily conflate the optimization objective with the surrounding training stack. In this repository, the surrounding stack is the main subject of the case study.

3 Full Reward Specification

Let c denote a completion, g the gold answer, $m \in \{\mathrm{num}, \mathrm{mc}\}$ the answer mode, and L the phase-specific completion budget. Here num denotes the numeric free-form path and mc the multi-choice path. The total per-sample reward is

$$R(c, g, m) = w_f R_{\text{format}} + w_p R_{\text{parseable}} + w_h R_{\text{finished}} + w_c R_{\text{correctness}} + w_b R_{\text{brevity}} + w_t R_{\text{tolerance}}. \tag{1}$$

Tag and parsing predicates. The implementation uses:

$$\mathrm{ReasoningTag}(c) = \mathbf{1}[\text{exactly one reasoning block exists}], \tag{2}$$

$$\mathrm{SolutionTag}(c) = \mathbf{1}[\text{exactly one solution block exists}], \tag{3}$$

$$\mathrm{Malformed}(c, m) = \begin{cases} \mathbf{1}[\text{numeric extraction or parsing fails}], & m = \mathrm{num}, \\ \mathbf{1}[\text{option extraction fails}], & m = \mathrm{mc}, \end{cases} \tag{4}$$

$$\mathrm{Finished}(c, m) = \begin{cases} 1, & \mathrm{ReasoningTag}(c) = 1 \wedge \mathrm{SolutionTag}(c) = 1 \\ & \wedge\ \mathrm{Malformed}(c, m) = 0, \\ 0, & \text{otherwise}. \end{cases} \tag{5}$$

Reward components. The six reward components are:

$$R_{\text{format}}(c,m) = \text{ReasoningTag}(c) + \text{SolutionTag}(c) - 0.5\,\text{Malformed}(c,m), \tag{6}$$

$$R_{\text{parseable}}(c,m) = \begin{cases} \mathbf{1}[\text{numeric parsing succeeds}], & m = \text{num}, \\ \mathbf{1}[\text{Malformed}(c,m) = 0], & m = \text{mc}, \end{cases} \tag{7}$$

$$R_{\text{finished}}(c,m) = \begin{cases} 1, & \text{Finished}(c,m) = 1 \wedge \text{tok}(c) < L, \\ -1, & \text{tok}(c) \geq L \wedge \text{Finished}(c,m) = 0, \\ 0, & \text{otherwise}, \end{cases} \tag{8}$$

$$R_{\text{correctness}}(c,g,m) = \mathbf{1}[\text{exact match}], \tag{9}$$

$$R_{\text{brevity}}(c) = \begin{cases} -1, & \text{tok}(c) \geq L, \\ 1, & \text{tok}(c)/L \leq 0.55, \\ 0.5, & 0.55 < \text{tok}(c)/L \leq 0.75, \\ 0, & 0.75 < \text{tok}(c)/L \leq 0.90, \\ -0.5, & 0.90 < \text{tok}(c)/L < 1, \end{cases} \tag{10}$$

$$R_{\text{tolerance}}(c,g,m) = \begin{cases} 1, & \text{tolerance is enabled, exact match is false,} \\ & \text{and the tolerance check holds,} \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$

Tolerance reward is active only in Phases C and D. Exact match suppresses tolerance reward, so the tolerance component acts strictly as a denser near-miss signal rather than an additional bonus for already-correct answers.

Phase-initialized weights. The configured initial weights are manual curriculum defaults rather than values learned by a meta-optimizer:

Phase	Format	Parseable	Finished	Correctness	Brevity	Tolerance
A	1.0	1.0	1.5	2.0	0.25	0.0
B	0.75	0.75	1.0	4.0	0.20	0.0
C	0.50	0.50	0.75	5.0	0.20	1.0
D	0.50	0.50	0.75	5.0	0.30	1.0

4 Evaluation Metrics and Checkpoint Scores

For each prompt i, the evaluator samples one or more completions $c_{i,j}$. It computes:

$$\text{EM} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{first sampled completion matches exactly}], \tag{12}$$

$$\text{TolAcc} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{first sampled completion is correct under tolerance}], \tag{13}$$

$$\text{ParseRate} = \frac{1}{M}\sum_{i,j} \mathbf{1}[\text{sample } (i,j) \text{ is parseable}], \tag{14}$$

$$\text{MalfRate} = \frac{1}{M}\sum_{i,j} \mathbf{1}[\text{sample } (i,j) \text{ is malformed}], \tag{15}$$

$$\text{TruncRate} = \frac{1}{M}\sum_{i,j} \mathbf{1}[\text{sample } (i,j) \text{ is truncated}]. \tag{16}$$

It also computes tag-compliance rates, completion-token averages, repetition rate, sampled-answer diversity, and mean logged reward components.

Table 1: Controller rules implemented in `staged_rl/controller.py`.

Rule	Trigger	Action
Parseable guard	parseable answer rate < 0.85	Raise parseable reward to at least 0.75
Format guard	solution tag < 0.90 or reasoning tag < 0.88 or malformed rate > 0.10	Raise format reward to at least 0.75
Finish guard	truncation rate > 0.15 or average completion tokens exceed $0.85L$	Add 0.25 to finished reward
Stable structure test	parseable ≥ 0.92, solution ≥ 0.95, reasoning ≥ 0.93, malformed ≤ 0.05, truncation ≤ 0.08	Enable plateau-based correctness escalation once enough history exists
Correctness escalation	Stable structure holds over a two-checkpoint window and exact-match gain is below 0.02	Add 0.50 to correctness reward

The checkpoint registry then defines three scores:

$$\begin{aligned} S_{\text{struct}} &= 0.35\,\text{ParseRate} + 0.20\,\text{SolutionTag} \\ &\quad + 0.20\,\text{ReasoningTag} - 0.15\,\text{MalfRate} \\ &\quad - 0.10\,\text{TruncRate}, \end{aligned} \tag{17}$$

$$S_{\text{corr}} = 0.70\,\text{EM} + 0.20\,\text{TolAcc} + 0.10\,\text{ParseRate}, \tag{18}$$

$$\begin{aligned} S_{\text{comp}} &= 0.45\,\text{EM} + 0.15\,\text{TolAcc} + 0.15\,\text{ParseRate} \\ &\quad + 0.10\,\text{SolutionTag} + 0.05\,\text{ReasoningTag} \\ &\quad - 0.05\,\text{MalfRate} - 0.05\,\text{TruncRate}. \end{aligned} \tag{19}$$

These scores back the aliases `best_structure`, `best_correctness`, and `best_composite`. The `latest` alias is purely chronological.

5 Controller Rules and Thresholds

The repository's controller audit covers 26 evaluated checkpoints. Across those checkpoints:

- parseability guard fires 2 times,
- formatting guard fires 2 times,
- finish guard fires 2 times,
- correctness escalation fires 12 times, and
- 6 correctness escalations occur when exact match has regressed rather than merely plateaued.

This behavior is consistent with the code: once the system judges structure to be stable, it reacts to weak exact-match progress by raising correctness pressure even if the immediate trend is negative.

6 Runtime Profile and Resource-Constrained Operation

The archived successful runs share the same low-memory profile:

- max sequence length reduced from 16384 to 1280,

(a) Primary hardware-profile changes. (b) Secondary and phase-specific changes.

Figure 3: Supporting figures for the repository's conservative Kaggle T4 profile.

- image size reduced from 512 to 336,
- LoRA rank reduced from 16 to 8,
- sampled generations reduced from 4 to 2,
- max prompt length reduced from 1024 to 320,
- max completion length reduced from 256 to 64 in the default Kaggle profile,
- max completion length increased to 96 only for Phase D,
- evaluation reduced to one sample per prompt and at most two examples per subset.

The repository interprets these changes as runtime stabilizers rather than task simplifiers. They reduce per-step VRAM pressure and evaluation overhead, but they also reduce reasoning budget, visual detail, and generation diversity.

7 Transfer and Continuation Semantics

The code distinguishes three different mechanisms that can be conflated in informal descriptions:

1. **Cross-notebook transfer.** A checkpoint directory is physically moved through a Kaggle dataset attachment or another external bundle.
2. **Warm start.** A fresh base model is loaded, a fresh LoRA adapter is attached, and adapter weights are copied from the selected checkpoint into that new adapter.
3. **Same-phase resume.** Full trainer state is restored from a checkpoint, including optimizer state, scheduler state, trainer progress, and controller state.

This distinction matters for the report's claims. Cross-phase continuation often uses alias-based warm start, not exact trainer resume. Same-phase restarts can use exact trainer resume if the selected checkpoint belongs to the current phase.

8 Additional Notes on Scope

The repository contains scaffolding for a multi-choice branch and an optional robustness stage, but neither path provides the main evidence used in the report. Similarly, the runner logs when multiple GPUs are visible but does not automatically switch to DDP or tensor parallelism. The strongest claims are therefore:

- about staged RL dynamics for numeric visual reasoning,
- about metric-gated control under non-monotonic checkpoint trajectories, and
- about making a single-GPU GRPO pipeline more auditable and reproducible.